

JDBC es una API estándar que permite lanzar consultas a una BD relacional. El desarrollador siempre trabaja contra los paquetes `java.sql` y `javax.sql`.

Para poder conectarse a la BD y lanzar consultas, es preciso tener un driver adecuado para ella.

- Un driver suele ser un fichero .jar que contiene una implementación de todas las interfaces de la API de JDBC.
- El driver lo proporciona el fabricante de la BD o un tercero.
- Nuestro código nunca depende del driver, dado que siempre trabaja contra los paquetes `java.sql` y `javax.sql`

Independencia de la BD

Idealmente, si nuestra aplicación cambia de BD, no necesitamos cambiar el código; simplemente, necesitamos otro driver.

Sin embargo, desafortunadamente las BBDD relacionales usan distintos dialectos de SQL.

[Ejemplo de actualización: es.udc.ws/jdbctutorial.InsertExample](http://es.udc.ws/jdbctutorial.InsertExample)

Ejecución de sentencias

Interfaz **Connection**

- Representa una conexión a la BD
- `prepareStatement`: permite construir objetos
- `PreparedStatement` para lanzar consultas.

Interfaz **PreparedStatement**

- Contiene la consulta SQL **parametrizada** que se va a lanzar.
- Parámetros: los caracteres "?" que aparecen en la consulta.
- Dispone de métodos `setXXX` para dar valor a los parámetros.
 - Los parámetros se enumeran de 1 en adelante.
- `executeUpdate`
 - Permite lanzar consultas SQL de actualización (e.g. `INSERT`, `UPDATE`, `DELETE`, etc)
 - Devuelve el número de filas afectadas por la actualización
- `executeQuery`
 - Permite lanzar consultas SQL de lectura (e.g. `SELECT`).
- Formateo de datos con `PreparedStatement`
 - Cuando se lanza la consulta, el driver sustituye y **formatea automáticamente** los parámetros en el formato requerido por la base de datos

- Cómodo: el driver formatea los datos, no el desarrollador
- Portable: el driver usará el formato adecuado para la base de datos subyacente
- Ejemplo 1: cadenas de caracteres y números enteros
 - En `InsertExample`, la consulta que se lanza en la primera iteración del bucle es `INSERT INTO TutMovie (movieId, title, runtime) VALUES ('movie-1', 'movie-1 title', 90)`
 - El driver entrecomilla los dos primeros valores (porque son de tipo cadena de caracteres) y no aplica ningún formato especial al tercer valor (porque es de tipo entero)

SQLException

Los métodos de la API de JDBC reportan cualquier error lanzando `SQLException` ("checked") o una de sus hijas

Procesamiento de filas resultado

La interfaz `ResultSet` representa todas las filas que han concordado con la consulta de búsqueda.

Iteración sobre las filas:

- La implementación de `ResultSet` mantiene un cursor, inicialmente posicionado antes de la primera fila.
- Si no quedan filas por leer, `next` devuelve `false`
- En otro caso, avanza el cursor en una posición y devuelve `true`
 Dispone de métodos `getXXX` para acceder a los valores de las columnas de la fila en la que está posicionado el cursor.

Obtención de conexiones

La clase `ConnectionManager` proporcionada en el ejemplo facilita la obtención de conexiones con la API básico de JDBC.

`java.sql.DriverManager` dispone del método estático `getConnection` que permite obtener una conexión a la BD a partir de:

- Una URL (formato especificado en la documentación del driver) que indica la máquina en la que corre la BD y el nombre del esquema.
- El identificador y contraseña de un usuario que tenga permisos de acceso.

En una aplicación real, para que no sea necesario modificar el código cuando se cambia la configuración de acceso a la base de datos, la URL, el usuario y la contraseña se deben leer de un fichero de configuración.

Liberación de recursos

Los ejemplos utilizan la construcción `try-with-resources` para abrir/cerrar las conexiones `try-with-resources`

- Es una extensión de la construcción `try-catch` introducida en Java 7
- Permite declarar uno o más "recursos" después de la palabra clave `try`
 - Los recursos se declaran entre paréntesis, separados por ",".
- Los "recursos" son objetos que implementan la interfaz `java.lang.AutoCloseable`, que sólo dispone del método `close`.
- Los recursos deben crearse cuando se declaran.
- El compilador de Java garantiza que se invocará al método `close` de los recursos declarados cuando se termine el bloque `try`, tanto si se producen excepciones como sino.
 - Cuando se ejecuten los posibles bloques `catch/finally`, los recursos declarados ya estarán cerrados.

Los ejemplos declaran el recurso "conexión JDBC" en la construcción `try-with-resources`

- `java.sql.Connection` extiende `java.lang.AutoCloseable`
- Cuando termina la ejecución del bloque `try`, el código generado por el compilador invoca al método `close` de la conexión.

Por otra parte, al margen de la construcción `try-with-resources`, el driver JDBC garantiza que

- Cuando se cierra una conexión, se cierran todos sus `PreparedStatement` asociados.
 - Cuando se cierra un `PreparedStatement`, se cierran todos sus `ResultSet` asociados.
- En consecuencia, "se cierra todo".

Si no se utiliza `try-with-resources`, tendríamos que cerrar la conexión explícitamente

```
Connection connection = null;

try
{
    connection = DriverManager.getConnection();
    // <<Interactuar con la BD>>
}
catch (Exception e)
{
    e.printStackTrace(System.err);
}
finally
{
    try
    {
```

```

        if(connection != null)
        {
            connection.close();
        }
    }
    catch (Exception e)
    {
        e.printStackTrace(System.err)
    }
}

```

finalize

- La implementación de la interfaz `Connection` debe redefinir `finalize` para que invoque a `close` en caso de que el desarrollador no lo haya hecho
 - `finalize` es un método definido en `Object`; el recolector de basura lo invoca antes de eliminar un objeto de memoria.
- En resumen, se crea la **ilusión** de que podríamos interactuar con la BD de la siguiente manera, es decir, sin cerrar la conexión explícitamente ni implícitamente vía `try-with-resources`:

```

try
{
    Connection connection = DriverManager.getConnection();
    // <<Interactuar con la BD>>
}
catch (Exception e)
{
    e.printStackTrace(System.err);
}

```

Sin embargo, en un caso real, no sería buena idea: Supongamos una aplicación servidora multi-thread, donde cada thread puede tener que acceder a la BD, un ejemplo de tal aplicación servidora es un servicio/aplicación Web Java

- Como veremos más adelante, los servicios/aplicaciones Web Java se ejecutan normalmente dentro de servidores de aplicaciones.
- Un servidor de aplicaciones tiene cada petición HTTP en un thread.
- Es posible atender múltiples peticiones concurrentemente.
- Además, un gestor de BD no puede tener abiertas más "N" conexiones.
- Si cada thread que accede a la BD, no cierra la conexión una vez termine su trabajo, la conexión no se cerrará hasta que el recolector de basura elimine esa conexión, que se ha

quedado sin referenciar.

- En un momento dado, puede ocurrir que se hayan procesado N peticiones HTTP y que sus respectivas conexiones todavía no hayan sido eliminadas por el recolector de basura.
 - El recolector de basura decide eliminar memoria cuando lo considera oportuno.
- Cuando llegue la siguiente petición, `DriverManager.getConnection` devolverá `SQLException` porque la BD no admite más conexiones.
 - Las "n" conexiones anteriores todavía no se han liberado, a pesar de que nadie las está usando.

Conclusión: Cada thread debe liberar la conexión inmediatamente una vez termine de interactuar con la BD, bien explícitamente o bien mediante `try-with-resources`.

Tipos SQL y Java

`ResultSet` y `PreparedStatement` proporcionan métodos `getXXX` y `setXXX`

¿Cuál es la correspondencia entre tipos Java y tipos SQL?

Idea básica: un dato de tipo Java se puede almacenar en una columna cuyo tipo SQL sea consistente con el tipo Java.

Tipo Java	Tipo SQL
boolean	BIT
byte	TINYIT
short	SMALLINT
int	INTEGER
long	BIGINT
float	REAL
double	DOUBLE
java.math.BigDecimal	NUMERIC
String	VARCHAR o LONGVARCHAR
byte[]	VARBINARY o LONGVARBINARY
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP

DataSources

Interfaz `javax.sql.DataSource`

- Entre otros, dispone del método `getConnection`
- Cuando se utiliza esta interfaz, el desarrollador no tiene que especificar la URL, el usuario y la contraseña para pedir la conexión.
Los servidores de aplicaciones Java y algunos frameworks ofrecen implementaciones de la interfaz `DataSource`
- A nivel de implementación utilizan `DriverManager.getConnection` para obtener las conexiones, aunque como veremos más adelante, la mejor estrategia puede ser compleja.
- Utilizar ficheros de configuración para especificar, como mínimo, la URL, el usuario y la contraseña.
- Tanto con Jetty como con Tomcat (servidores de aplicaciones) configuraremos objetos `DataSource` para acceder a la BD

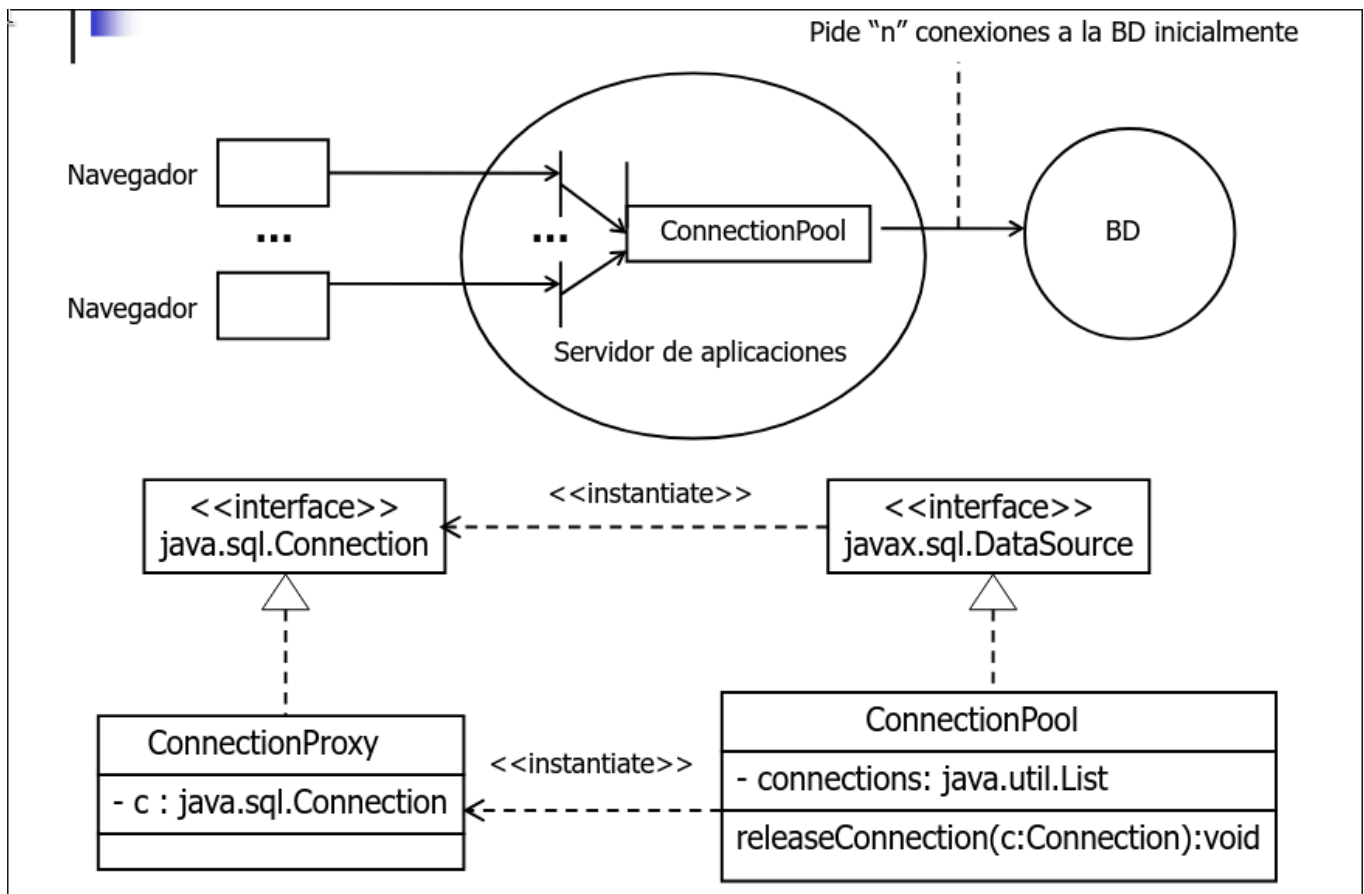
Pool de conexiones

Problema:

- Servidor de aplicaciones que recibe muchas peticiones HTTP por minuto.
- Es posible pedir una conexión a la BD con `DriverManager.getConnection` o el método `getConnection` de un objeto `DataSource`
- `DriverManager.getConnection` pide una conexión directamente a la BD
 - Es una operación lenta -> se convierte en cuello de botella.
- Es una implementación básica de `DataSource`, el método `getConnection` también invoca `DriverManager.getConnection`
- Además, con cualquiera de los dos métodos, si en ese momento la BD ya no admite más conexiones, porque supera el máximo permitido, los métodos `getConnection` devuelven una excepción

Solución: Pool de conexiones

- Los servidores de aplicaciones Java proporcionan implementaciones de `DataSource` que utilizan la estrategia **pool de conexiones**
- El objeto `DataSource` gestiona un conjunto de conexiones que previamente ha solicitado la BD
- El desarrollador sólo trabaja contra la interfaz `DataSource`
 - La estrategia es transparente al desarrollador.



ConnectionPool

- Cuando se crea, pide N conexiones a la BD, usando `DriverManager.getConnection` y ñas almacena en una lista.
- `getConnection`
 - Si quedan conexiones libres en la lista, elige una, la marca como **usada**, y devuelve un objeto `ConnectionProxy` que la contiene.
 - En otro caso, deja durmiendo (`wait`) al thread llamador.
- `releaseConnection`
 - Devuelve la conexión a la lista, la marca como **libre**, y la notifica (`notifyAll`) a los posibles thread que esperan por una conexión.
- `ConnectionProxy`
 - Proxy de la conexión real
 - `close` usa `releaseConnection` para devolver la conexión real al pool.
 - `finalize` si no se ha llamado a `ConnectionProxy.close`, lo llama
 - El resto de operaciones delegan en la conexión real.

Observaciones:

- Cuando el desarrollador invoca `getConnection` sobre el objeto `DataSource`
 - Si hay una conexión libre -> se le devuelve rápidamente de la lista, no se accede a BD.
 - Si no hay ninguna conexión libre -> el thread llamador se queda dormido hasta que haya una.
- Las conexiones reales no se cierran.

Caídas de la BD

- Si la BD se cae, las conexiones del pool se invalidan aunque se vuelva a rearrancar la BD, porque los sockets subyacentes ya no son válidos.
- Para hacer frente a este problema, la implementación de `getConnection` puede comprobar si la conexión que devuelve es correcta (**está viva**).
 - Opción 1: haciendo uso de la API especificada del fabricante del driver.
 - Opción 2: lanzando una consulta poco costosa a la BD (si no se produce una `SQLException`, la conexión es correcta).

Configuración del pool

- Además de la configuración básica de un `DataSource`, se puede especificar el número de conexiones a la BD que se solicitan inicialmente, la consulta de comprobación de conexión viva (si se requiere), etc.

Transacciones

Permiten ejecutar bloques de código con las propiedades ACID (Atomicity-Consistency-Isolation-Durability), por defecto, cuando se crea una conexión está en modo auto-commit, cada consulta lanzada se ejecuta en su propia transacción. Para ejecutar varias consultas en una misma transacción es preciso deshabilitar el modo auto-commit de la conexión, lanzar las consultas y terminar con `connection.commit()` si todo va bien, o `connection.rollback()` en otro caso.

[TransactionExample](#)

Transaction isolation levels

`java.sql.Connection` proporciona el método `setTransactionIsolation`, que permite especificar el nivel de aislamiento deseado.

- `TRANSACTION_NONE`: transacciones no soportadas

- `TRANSACTION_READ_UNCOMMITTED` : pueden ocurrir "dirty read", "non-repeatable reads" y "phantom reads"
- `TRANSACTION_READ_COMMITTED` : pueden ocurrir "non-repeatable reads" y "phantom reads"
- `TRANSACTION_REPEATABLE_READ` : pueden ocurrir "phantom reads"
- `TRANSACTION_SERIALIZABLE` : elimina todos los problemas de concurrencia.

Mayor nivel de aislamiento -> la BD realiza más bloqueos -> menos concurrencia. Por sencillez, en la asignatura realizaremos las transacciones con el nivel de aislamiento

`TRANSACTION_SERIALIZABLE` .

Existen técnicas que permiten trabajar transaccionalmente en muchas situaciones con un nivel de aislamiento inferior

- Menos bloqueos en la BD
- En "Programación Avanzada" se estudia la técnica de "Optimistic Locking"